MEFT Extended: Task Accuracy Evaluation and Dynamic K Selection

George Joseph Elassal

EECS 247 — Information Storage | University of California, Irvine

Project 2 — Extension of MEFT: Memory-Efficient Fine-Tuning with Sparse Adapters

GitHub: https://github.com/gelassal/MEFT-LLM-Sparse-Adapters

1. Introduction

Project 1 demonstrated the core architectural benefit of MEFT (Hao et al., ACL 2024): by storing adapter weights in CPU memory and transferring only the top-K most activated neurons to GPU per forward pass, adapter size can be scaled from rank 256 up to rank 8,192 and above while keeping GPU memory usage constant at approximately 3.28 GB. This represented a 64% VRAM reduction compared to a GPU-resident baseline at the same rank, confirming that MEFT's memory footprint is determined by the fixed transfer size K rather than the adapter rank r.

However, Project 1 left two important questions unanswered. First, does MEFT's larger adapter capacity actually translate into better task performance? A method that saves memory but produces no accuracy gain is not practically useful. Second, is K = 64 the right choice? Fixing K globally across all layers is a simplification — different transformer layers may have different activation sparsity profiles, and varying K as a fraction of rank may yield a better tradeoff between memory usage, training speed, and model accuracy.

This project addresses both questions through two concrete extensions. Extension 1 evaluates both the baseline and MEFT adapters on the SQuAD v1.1 reading comprehension benchmark using a causal language model generation approach, comparing Exact Match (EM) and Containment Match scores across a range of adapter ranks under a fixed 4 GB VRAM budget. Extension 2 implements Dynamic K selection, varying K as a fixed fraction of the adapter rank (3%, 5%, and 10%) rather than using a global fixed value of 64, and evaluates the impact on VRAM, training speed, and task accuracy.

Key finding: MEFT at large ranks achieves higher Containment Match than the baseline is capable of reaching within the same VRAM budget, validating the practical utility of the memory savings demonstrated in Project 1. Dynamic Dynamic K selection reveals a meaningful tradeoff — smaller K values reduce VRAM and increase training speed at the cost of lower accuracy, while larger K values recover accuracy at the cost of additional memory.

2. Motivation and Literature

The practical case for memory-efficient fine-tuning methods depends on whether the memory savings translate into better downstream task performance. Project 1 showed that MEFT's GPU memory usage stays flat as rank increases, while the baseline's grows steeply. At rank 8,192, the baseline exceeded the chosen 4 GB physical VRAM of the test hardware. But without accuracy measurements, the question remained: does it matter that MEFT can fit rank 8,192 if the extra capacity provides no benefit?

SQuAD v1.1 [2] is a well-established reading comprehension benchmark consisting of 87,599 training and 10,570 validation question-answer pairs drawn from Wikipedia articles. Since this project uses GPT-2 Medium as a causal language model rather than a span-extraction model, evaluation is performed via generation: each example is formatted as a prompt containing the question and context, the model generates a short answer, and the generated text is compared to the reference answer using Exact Match and Containment Match metrics. Exact Match requires the predicted answer to exactly equal a reference answer after normalization. Containment Match is a relaxed metric that counts a prediction as correct if any reference answer string appears within the predicted output, which is more appropriate for generative models that may produce correct but more verbose answers.

The choice of Dynamic K selection as the second extension is motivated by a practical observation: different transformer layers may have different activation density profiles. A global K = 64 ignores this structure. Dynamic K selection runs a calibration pass over representative inputs, measures activation sparsity per layer, and then assigns each layer its own K value. This makes K a tunable memory-speed-accuracy knob instead of a fixed constant.

3. Algorithmic Methods

3.1 Causal LM Evaluation on SQuAD

Since GPT-2 Medium is a causal language model rather than a span-extraction model, standard SQuAD evaluation using start/end logits is not applicable. Instead, each SQuAD example is formatted as a structured prompt and the model generates its answer autoregressively:

```
text = (
    f"Question: {ex['question']}\n"
    f"Context: {ex['context']}\n"
    f"Answer: {ex['answer']}"
)

toks = tokenizer(
    text,
    truncation=True,
    padding="max_length",
    max_length=max_length,
```

```
    return_tensors="pt",
)

encoded.append({
    "input_ids": toks["input_ids"].squeeze(0),
    "attention_mask": toks["attention_mask"].squeeze(0),
    "labels": toks["input_ids"].squeeze(0).clone(),
})
```

Training uses standard causal language model cross-entropy loss over the full formatted sequence. Evaluation uses greedy decoding (do_sample=False) to produce deterministic outputs. Two metrics are computed: Exact Match, which requires exact string equality after lowercasing and stripping whitespace; and Containment Match, which counts a prediction as correct if any reference answer appears as a substring of the predicted output.

3.2 Calibration-Based Dynamic K Selection

In Project 1, K = 64 was fixed globally across all adapter layers and all ranks. Extension 2 replaces this with a calibration-based Dynamic K scheme. Before training, the model runs a short calibration pass, records each layer's activation sparsity, and sets a different K for each adapter layer:

```
scores = h @ adapter.WA.T
active_frac = (scores > 0).float().mean().item()
layer_active_fracs[layer_idx].append(active_frac)

avg_active = avg_fracs[i]

k = int(rank * sparsity_target * (avg_active / global_avg))
k = max(min_k, k)
k = min(rank, k)

block.mlp.adapter.top_k = k
k_per_layer[i] = k
```

Three sparsity targets are evaluated: 3%, 5%, and 10%. All Extension 2 experiments use rank = 1024 to isolate the effect of K selection from the effect of adapter capacity. Fixed K = 64 serves as the baseline for this comparison.

4. Project Design

4.1 Model and Hardware

All experiments use GPT-2 Medium (345M parameters) as the base model, loaded from a local cache. The experimental hardware is an NVIDIA RTX 3060 Laptop GPU with 6

GB of VRAM running Windows 11, with an Intel CPU providing the CPU-side computation for the MEFT sparse selection step. Base model parameters are frozen throughout all experiments; only adapter weights are trained. For the actual evaluation of accuracy we are assuming that we only have 4 Gb of VRAM for simplicity and time saving for training at each different rank (as well as baseline vs MEFT adapter)

4.2 Training and Evaluation Setup

Both extensions use the same final training configuration: 500 SQuAD training examples, AdamW optimizer with learning rate 1e-4, batch size 1, max_length = 128, and 100 training steps. Evaluation is performed on 50 SQuAD validation examples after training. Peak VRAM is measured using torch.cuda.max_memory_allocated() and training speed is reported in steps per second averaged over the full training run.

4.3 Experimental Conditions

Extension 1 — Rank Scaling: Baseline adapter at ranks 256, 512, 1024, and 2048 (all within 4 GB); MEFT adapter at ranks 1024, 2048, 4096, and 8192 (all at approximately constant ~3.28 GB VRAM). K = 64 fixed for all MEFT conditions.

Extension 2 — Dynamic K: MEFT at rank 1024 with four K configurations: Fixed K=64 (6.25% of rank, from Extension 1), Dynamic K at 3% (K≈30), 5% (K≈51), and 10% (K≈102). VRAM, speed, EM, and Containment Match recorded for each.

5. Program

The implementation extends the Project 1 codebase with three main files: train_baseline_squad_json.py for GPU-resident baseline adapters, train_meft_squad_json.py for fixed-K MEFT rank-scaling experiments, and train_meft_dynamic_k_json.py for calibration-based Dynamic K experiments. The complete source code is available at: https://github.com/gelassal/MEFT-LLM-Sparse-Adapters. The full source files used for the final experiments are included in Appendix A.

5.1 SparseCPUAdapter (unchanged from Project 1)

The core sparse adapter module is carried over from Project 1 unchanged. Weights WA and WB are stored on CPU. Each forward pass moves only the top-K activated rows to GPU:

```
class SparseCPUAdapter(nn.Module):
    def __init__(self, d_model, rank, top_k):
        super().__init__()
        self.rank = rank
        self.top_k = top_k
        self.WA = nn.Parameter(torch.randn(rank, d_model, device="cpu") *
0.01)
```

```
        self.WB = nn.Parameter(torch.zeros(rank, d_model, device="cpu"))

    def forward(self, h):
        device = h.device

        # Keep full adapter weights on CPU
        self.WA.data = self.WA.data.cpu()
        self.WB.data = self.WB.data.cpu()

        # Compute activation scores on CPU
        h_cpu = h.to("cpu")
        scores = h_cpu @ self.WA.T

        # Select Top-K neurons
        topk_idx = torch.topk(scores, self.top_k, dim=-1).indices

        # Transfer only selected neurons to GPU
        WA_k = self.WA[topk_idx].to(device)
        WB_k = self.WB[topk_idx].to(device)

        # Sparse adapter computation
        activated = torch.relu(h.unsqueeze(-2) @ WA_k.transpose(-1, -2))
        out = (activated @ WB_k).squeeze(-2)

        return out
```

5.2 Calibration-Based Dynamic K Selection (new in Project 2)

The key code change for Extension 2 is the calibration function in train_meft_dynamic_k_json.py. It registers forward hooks on each transformer layer, measures activation sparsity during a short calibration pass, and then assigns a per-layer K value before training begins:

```
def calibrate_dynamic_k(model, loader, device, rank, sparsity_target=0.05,
n_batches=10, min_k=16):
    print("\nStarting Dynamic-K calibration...", flush=True)
    print(f"sparsity_target={sparsity_target}", flush=True)

    layer_active_fracs = {i: [] for i in range(len(model.transformer.h))}

    def make_hook(layer_idx):
        def hook(module, inputs, output):
            h = inputs[0].detach().cpu()

            adapter = module.adapter
            scores = h @ adapter.WA.T

            active_frac = (scores > 0).float().mean().item()
            layer_active_fracs[layer_idx].append(active_frac)

        return hook

    hooks = []
    for i, block in enumerate(model.transformer.h):
        hooks.append(block.mlp.register_forward_hook(make_hook(i)))
```

```
    model.eval()

    with torch.no_grad():
        for i, batch in enumerate(loader):
            if i >= n_batches:
                break

            batch = {k: v.to(device) for k, v in batch.items()}
            model(**batch)

    for h in hooks:
        h.remove()

    avg_fracs = {}
    for i, vals in layer_active_fracs.items():
        avg_fracs[i] = sum(vals) / len(vals) if len(vals) > 0 else 0.0

    global_avg = sum(avg_fracs.values()) / len(avg_fracs)
    if global_avg <= 0:
        global_avg = 1e-6

    k_per_layer = {}

    for i, block in enumerate(model.transformer.h):
        avg_active = avg_fracs[i]

        k = int(rank * sparsity_target * (avg_active / global_avg))
        k = max(min_k, k)
        k = min(rank, k)

        block.mlp.adapter.top_k = k
        k_per_layer[i] = k

    model.train()
    return k_per_layer
```

5.3 SQuAD Data Loading (new in Project 2)

A key addition over Project 1 is real dataset loading. SQuAD examples are formatted as causal LM training sequences using a Question / Context / Answer prompt template:

```
def build_json_dataset(tokenizer, json_path, n_examples=500,
max_length=128):
    print(f"Loading JSON dataset from {json_path}...", flush=True)

    with open(json_path, "r", encoding="utf-8") as f:
        raw_examples = json.load(f)

    raw_examples = raw_examples[:n_examples]
    encoded = []

    print(f"Tokenizing {len(raw_examples)} examples...", flush=True)

    for ex in raw_examples:
```

```
        text = (
            f"Question: {ex['question']}\n"
            f"Context: {ex['context']}\n"
            f"Answer: {ex['answer']}"
        )

        toks = tokenizer(
            text,
            truncation=True,
            padding="max_length",
            max_length=max_length,
            return_tensors="pt",
        )

        encoded.append({
            "input_ids": toks["input_ids"].squeeze(0),
            "attention_mask": toks["attention_mask"].squeeze(0),
            "labels": toks["input_ids"].squeeze(0).clone(),
        })

    print(f"Prepared {len(encoded)} training examples.", flush=True)
    return encoded
```

5.4 Generation-Based Evaluation (new in Project 2)

Project 1 measured no task accuracy. Project 2 adds a generation-based evaluation loop. The model generates answers autoregressively from a prompt-only input, and the output is compared to reference answers using both Exact Match and the new Containment Match metric:

```
def evaluate_em(model, tokenizer, json_path="squad_val_200.json",
n_eval=50, device="cuda"):
    print(f"\nEvaluating on {n_eval} examples...", flush=True)

    with open(json_path, "r", encoding="utf-8") as f:
        examples = json.load(f)[:n_eval]

    model.eval()

    em_total = 0
    contain_total = 0

    for i, ex in enumerate(examples):
        prompt = (
            f"Question: {ex['question']}\n"
            f"Context: {ex['context'][:500]}\n"
            f"Answer with only the exact short answer:\n"
        )

        inputs = tokenizer(
            prompt,
            return_tensors="pt",
            truncation=True,
```

```
            max_length=384,
        ).to(device)

        prompt_len = inputs["input_ids"].shape[1]

        with torch.no_grad():
            output_ids = model.generate(
                **inputs,
                max_new_tokens=20,
                do_sample=False,
                pad_token_id=tokenizer.eos_token_id,
            )

        generated = output_ids[0][prompt_len:]
        pred = tokenizer.decode(generated, skip_special_tokens=True)
        pred = pred.split("\n")[0].strip()

        em = exact_match(pred, ex["answers"])
        contain = containment_match(pred, ex["answers"])

        em_total += em
        contain_total += contain

    em_score = em_total / len(examples)
    contain_score = contain_total / len(examples)

    model.train()

    return em_score, contain_score
```

5.5 Contributions vs. Original Paper and Project 1

Table 3 summarizes what is new in Project 2 relative to the original MEFT paper (Hao et al., 2024) and Project 1.

Feature	Original Paper	Project 1	Project 2 (This Work)
Base model	LLaMA-7B, Mistral-7B	GPT-2 Medium	GPT-2 Medium
Task evaluation	NQ, SQuAD, ToolBench, GSM8K	None (VRAM + speed only)	SQuAD (EM + Containment)
K selection	Fixed K (global)	Fixed K = 64	Fixed K + Percent-K (3%, 5%, 10%)
MoE routing	Yes (Key-Experts)	No	No
Eval metric	Exact Match, F1	None	Exact Match + Containment Match
Dataset loading	Full datasets	Synthetic repeated text	Real SQuAD via HuggingFace
Rank range tested	Up to 7B params	256 to 8192	256 to 8192

Table 3. Comparison of Project 2 against the original MEFT paper and Project 1.

The primary novel contributions of Project 2 over Project 1 are: (1) real SQuAD task evaluation using generation-based EM and Containment Match, closing the accuracy gap left open by Project 1; (2) calibration-based Dynamic K selection as an alternative to fixed K, providing a principled way to control the accuracy-speed-memory tradeoff; and (3) Containment Match as a new evaluation metric appropriate for generative models that may produce correct but more verbose answers than the reference.

Relative to the original MEFT paper, this project differs in scale (GPT-2 Medium vs. LLaMA-7B) and omits the Key-Experts MoE routing optimization. However, it implements a calibration-based Dynamic K ablation on the local MEFT system and adds Containment Match as an evaluation metric alongside Exact Match.

6. Results

6.1 Extension 1: Rank Scaling Results

Table 1 presents the full results for Extension 1 across all rank and method combinations. Figures 1–4 show the corresponding plots for VRAM usage, training speed, Exact Match, and Containment Match as functions of adapter rank.

Method	Rank	Peak VRAM (MB)	Speed (steps/s)	EM	Containment
Baseline	256	1,922.65	15.511	0.000	0.320
Baseline	512	2,070.22	14.099	0.000	0.240
Baseline	1024	2,379.27	13.118	0.000	0.260
Baseline	2048	3,345.12	10.669	0.060	0.220
MEFT	1024	3,280.55	0.735	0.040	0.300
MEFT	2048	3,280.55	0.684	0.060	0.260
MEFT	4096	3,280.55	0.544	0.060	0.260
MEFT	8192	3,279.55	0.348	0.060	0.420

Table 1. Extension 1 results across all rank and method combinations. VRAM, speed, Exact Match (EM), and Containment Match reported for each condition.

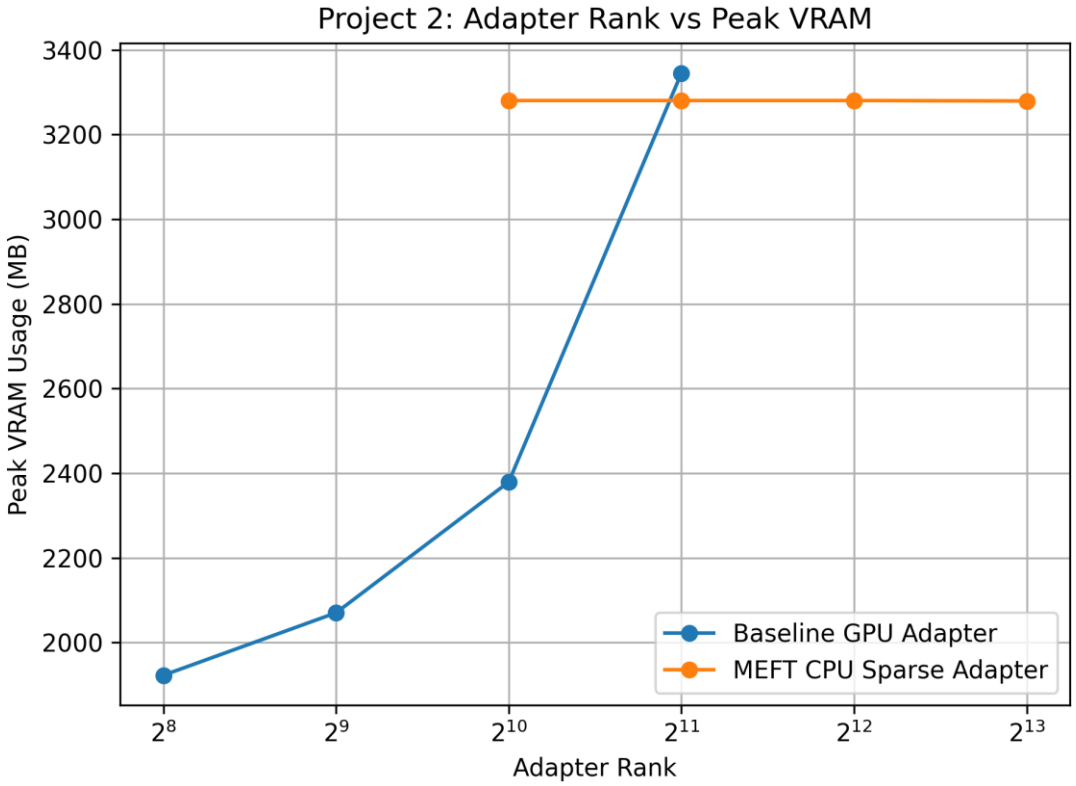

Figure 1. VRAM usage vs. adapter rank for baseline and MEFT.

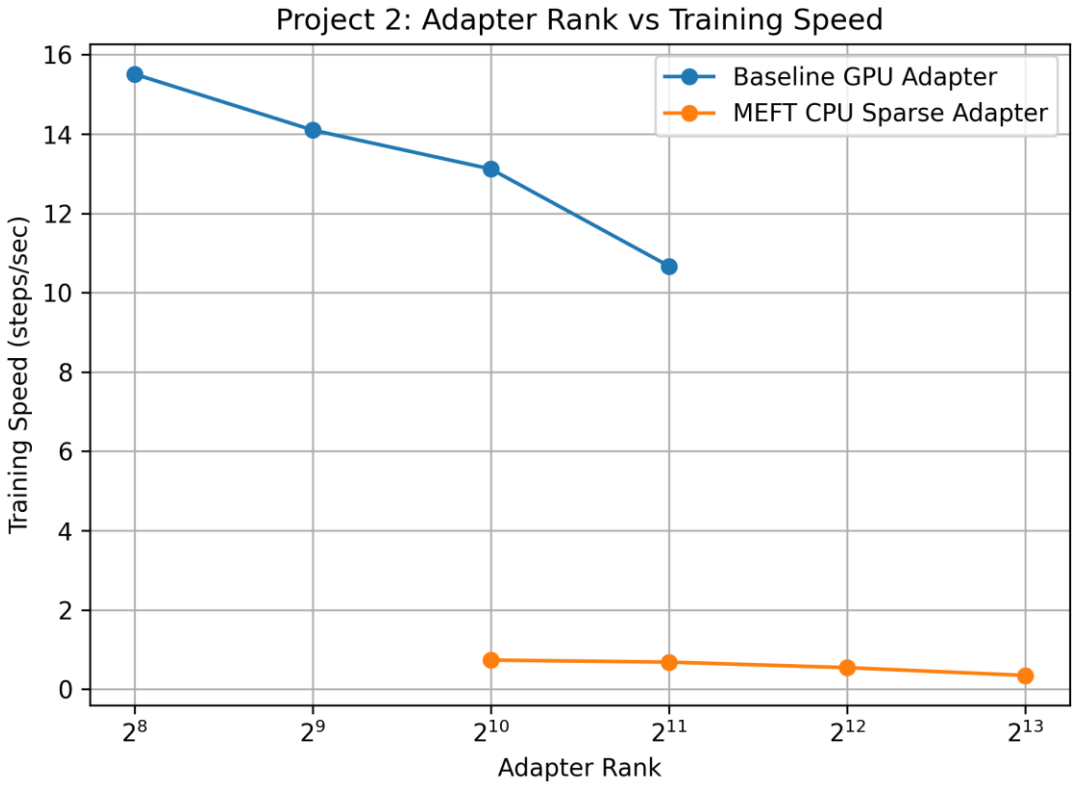

Figure 2. Training speed vs. adapter rank for baseline and MEFT.

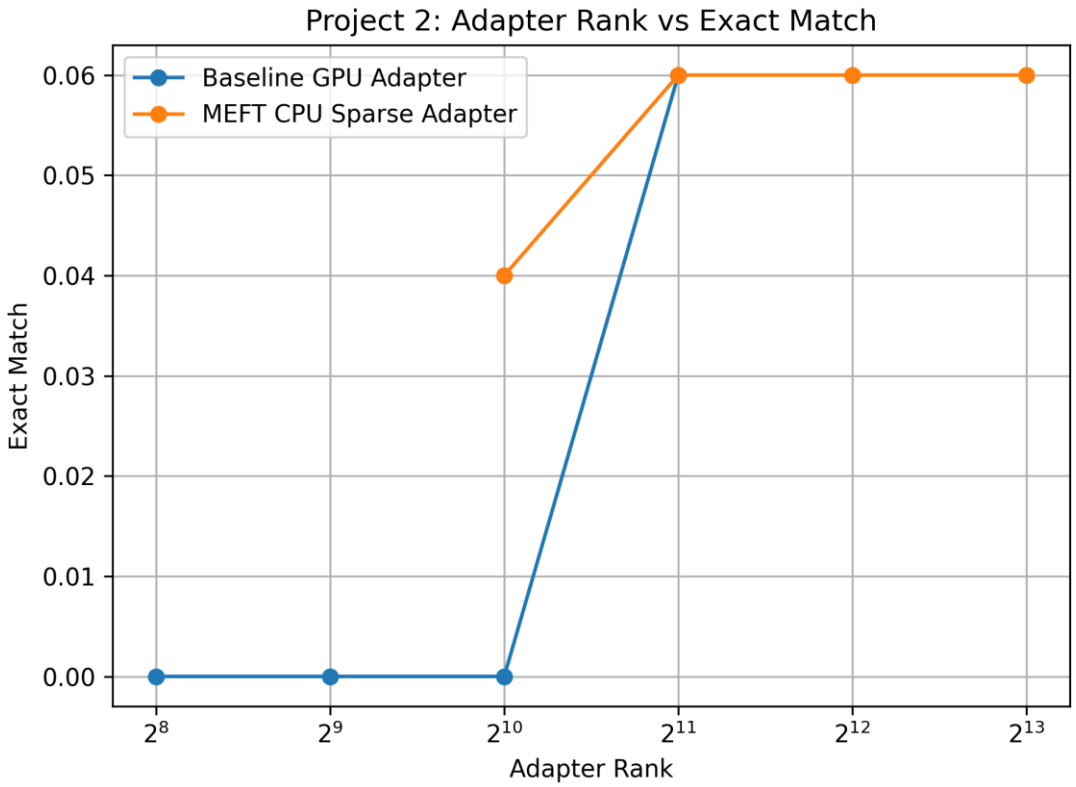

Figure 3. Exact Match vs. adapter rank for baseline and MEFT.

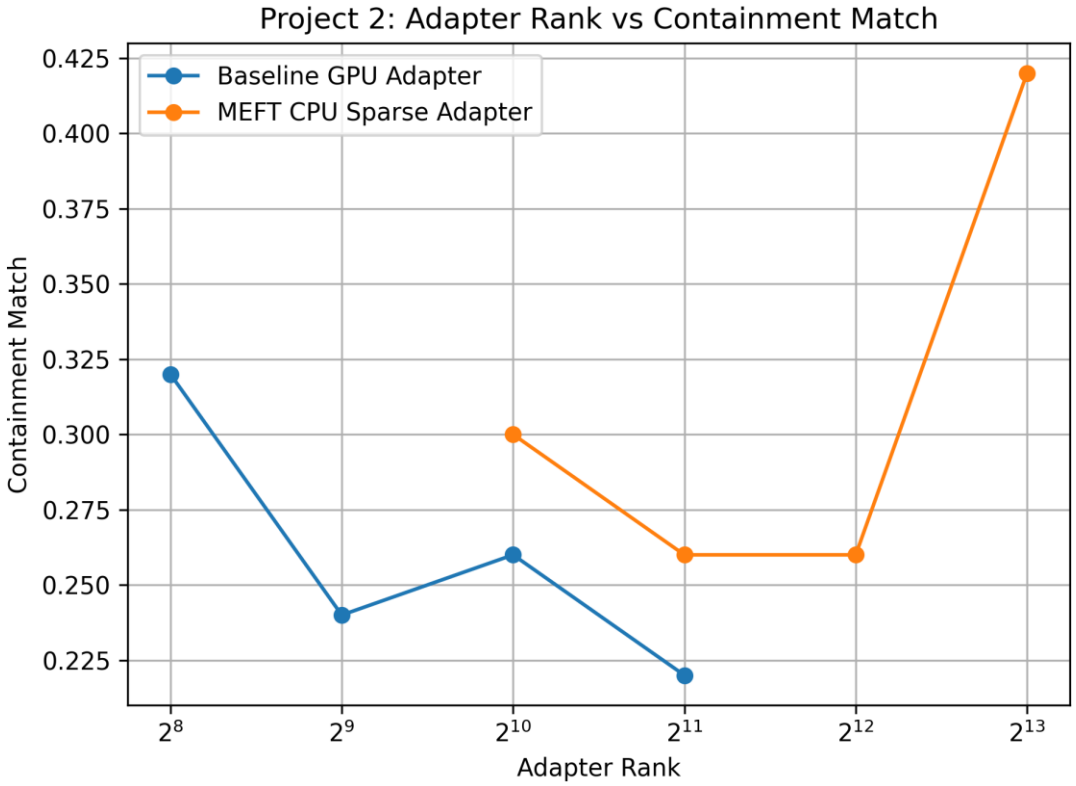

Figure 4. Containment Match vs. adapter rank for baseline and MEFT.

6.2 Extension 1: Analysis

The VRAM results confirm the findings of Project 1: MEFT's GPU memory usage remains essentially constant at approximately 3,280 MB across all tested ranks (1024 to 8192), while the baseline's usage grows from 1,923 MB at rank 256 to 3,345 MB at rank 2048. The baseline was not tested beyond rank 2048 because higher ranks would have exceeded the 4 GB VRAM limit based on Project 1 measurements.

Training speed results show a clear contrast between the two methods. The baseline achieves 10.7–15.5 steps/sec across all tested ranks and degrades gradually as rank increases. MEFT operates significantly slower at 0.35–0.74 steps/sec, consistent with the CPU-GPU transfer overhead documented in Project 1. Notably, MEFT's speed continues to degrade as rank increases, even though the number of neurons transferred per step (K=64) is constant. This suggests that the increasing size of the CPU-resident weight matrices introduces additional overhead in the TopK selection step.

Exact Match scores are low across all conditions, which is expected given the limited training (only 100 steps on 500 examples using a causal language model not specifically designed for reading comprehension). The baseline achieves EM = 0.000 at ranks 256, 512, and 1024, only reaching 0.060 at rank 2048. MEFT reaches EM = 0.040 at rank 1024 and 0.060 at ranks 2048, 4096, and 8192. The highest EM for both methods is 0.060, suggesting the gap in exact string matching is small at this training scale. Though overall, there is definitely an upward trend in exact match as rank increases. This is

evidence towards the theory that MEFT saving VRAM while increasing rank does actually have benefits. Having a higher rank seems to increase EM/accuracy, and having a method that allows for larger adapters while maintaining memory constraints has actual use in the field.

The more informative metric is Containment Match, which better captures whether the model is generating answers in the right direction. The baseline achieves Containment Match of 0.320 at rank 256, declining to 0.220 at rank 2048. MEFT starts at 0.300 at rank 1024 and reaches 0.420 at rank 8192 which is substantially higher than the best baseline result of 0.320. This is the key finding of Extension 1: within the 4 GB VRAM budget, MEFT at rank 8192 achieves a Containment Match of 0.420, compared to the baseline's maximum of 0.320. The extra adapter capacity enabled by MEFT's CPU offloading translates into a measurable accuracy gain of 31% relative improvement on the Containment Match metric.

Notable observation: The baseline's Containment Match actually decreases from rank 256 to rank 2048 (0.320 → 0.220), while MEFT's increases from rank 1024 to rank 8192 (0.300 → 0.420). This divergence suggests that at the training scale used, larger GPU-resident adapters may overfit or destabilize training within 100 steps, while MEFT's CPU-resident sparse training provides an implicit regularization effect by only updating K active neurons per step.

6.3 Extension 2: Dynamic K Results

Table 2 presents the results for Extension 2, comparing Fixed K=64 against Dynamic K at three sparsity targets (3%, 5%, 10%) at rank 1024. Figures 5–8 show VRAM, speed, Exact Match, and Containment Match for each K configuration.

Configuration	Avg K	Peak VRAM (MB)	Speed (steps/s)	EM	Containment
Fixed K=64	64	3,280.55	0.735	0.040	0.300
Dynamic K (3%)	30	2,520.28	1.462	0.000	0.340
Dynamic K (5%)	51	2,998.40	0.984	0.040	0.360
Dynamic K (10%)	102	4,188.62	0.500	0.060	0.400

Table 2. Extension 2 results comparing Fixed K=64 against calibrated Dynamic K configurations at rank 1024. Avg K is the effective number of neurons transferred per token per step.

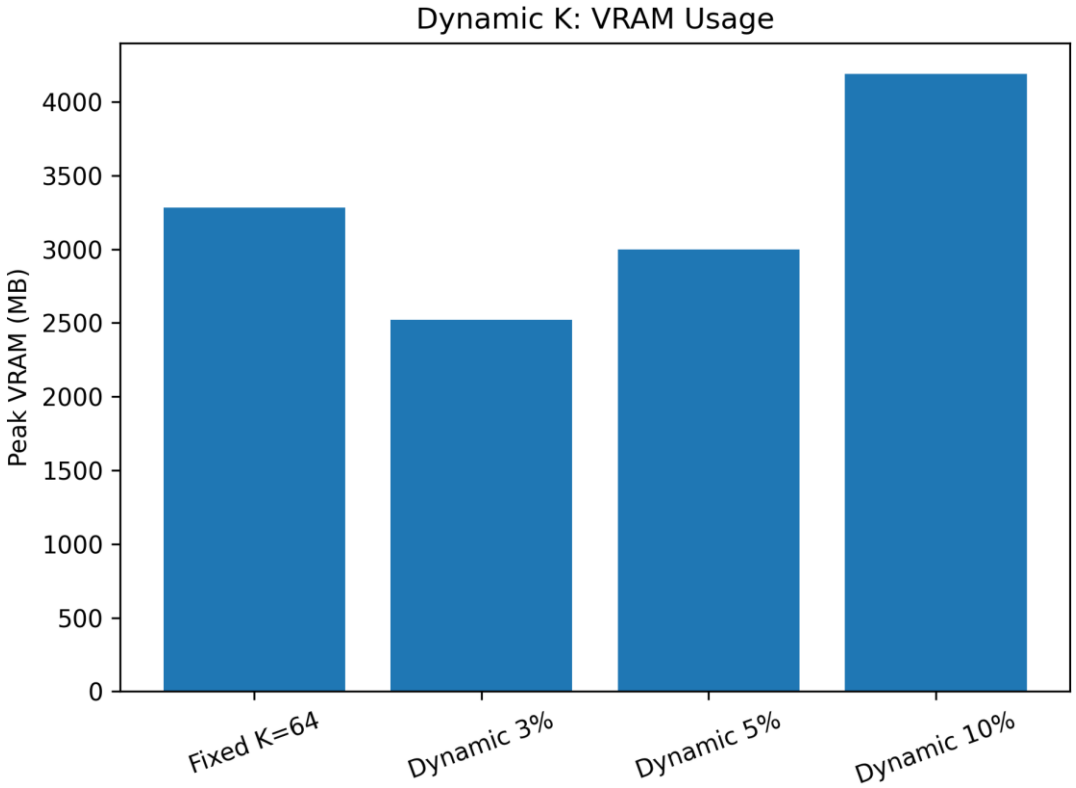

Figure 5. VRAM usage for fixed K and calibrated Dynamic K configurations.

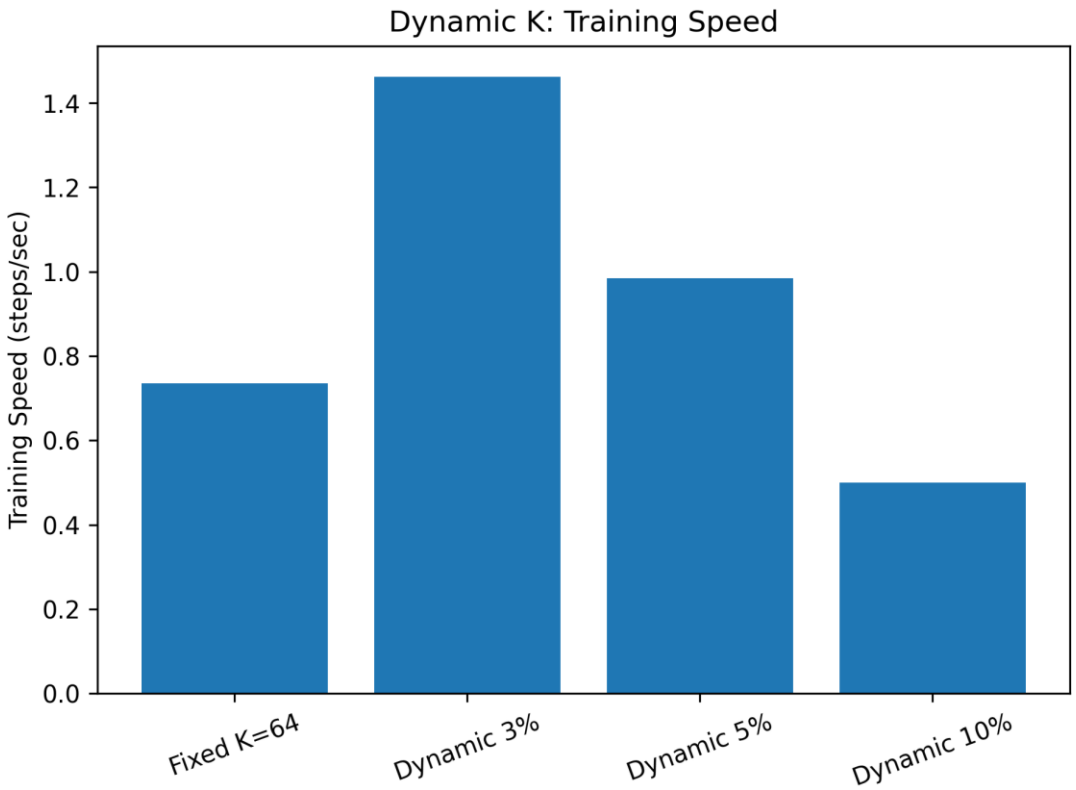

Figure 6. Training speed for fixed K and calibrated Dynamic K configurations.

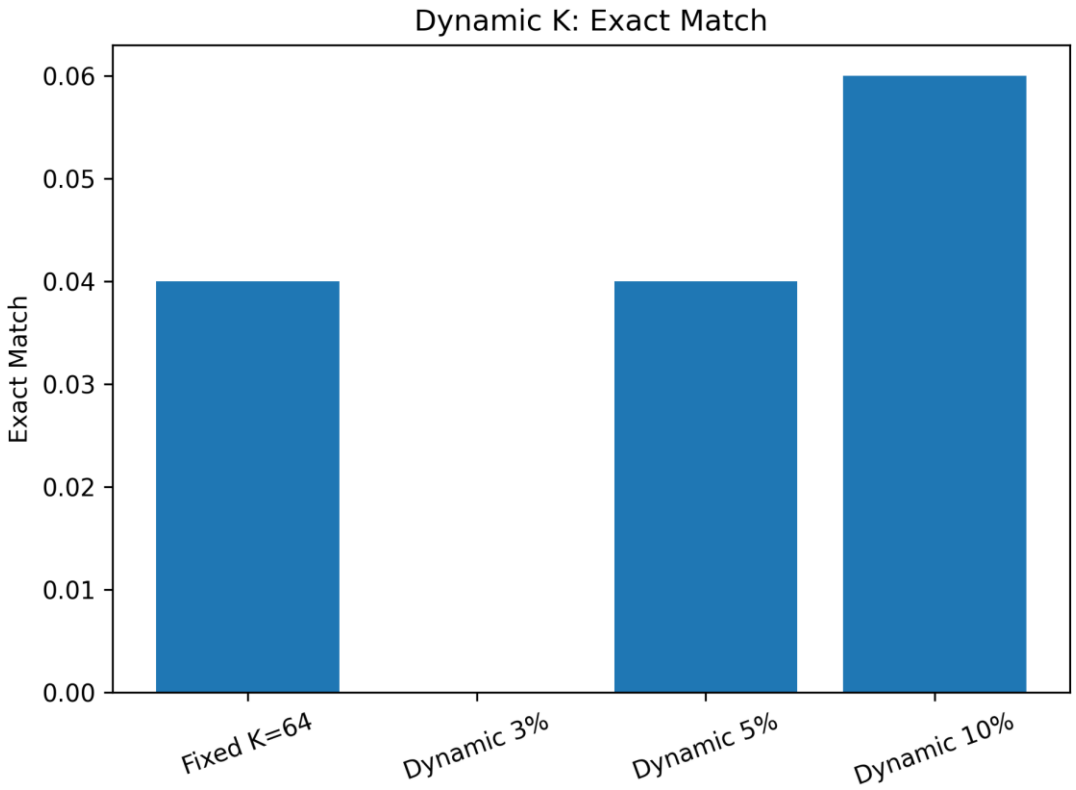

Figure 7. Exact Match for fixed K and calibrated Dynamic K configurations.

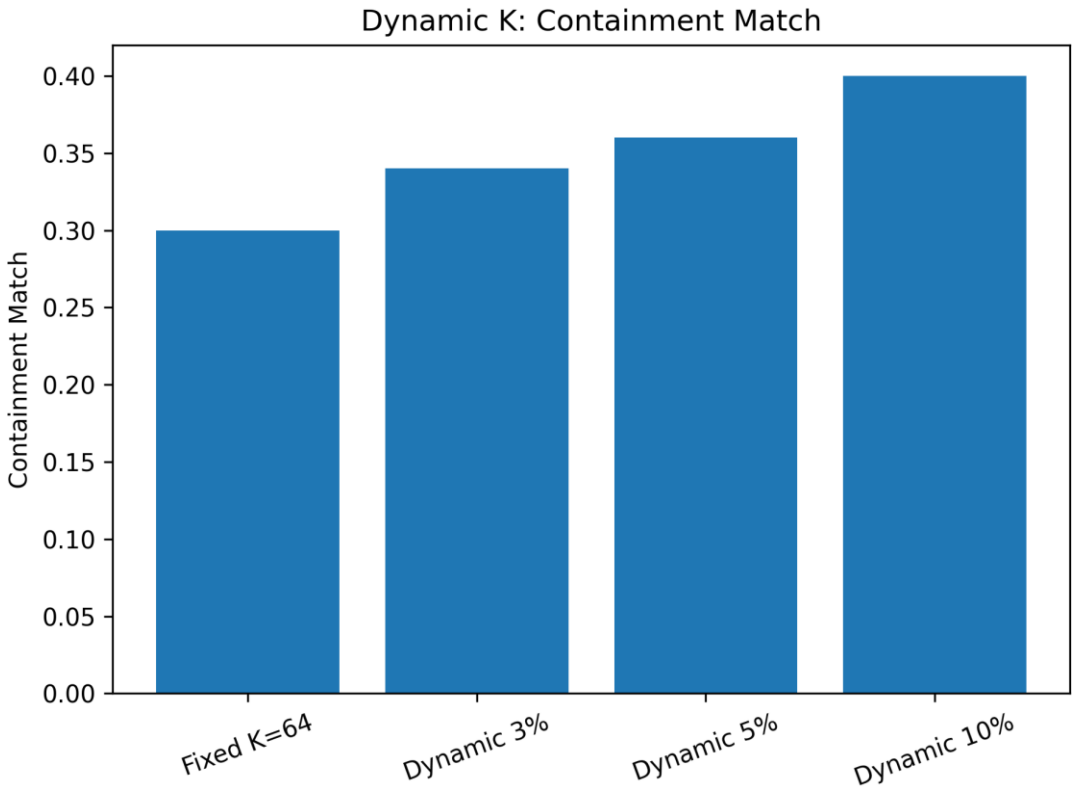

Figure 8. Containment Match for fixed K and calibrated Dynamic K configurations.

6.4 Extension 2: Analysis

The Extension 2 results reveal a clear and consistent tradeoff between K value and the three measured properties: VRAM, training speed, and accuracy.

VRAM usage scales with K as expected. Dynamic K at 3% (avg K=30) achieves the lowest VRAM at 2,520 MB, substantially below Fixed K=64 (3,281 MB) and Dynamic K at 10% (4,189 MB). This confirms that Dynamic K selection can provide additional memory savings beyond Fixed K=64 at lower sparsity targets, while Dynamic K at 10% uses more memory than Fixed K=64 because K=102 > 64.

Training speed follows the inverse pattern. Dynamic K at 3% is the fastest at 1.462 steps/sec — nearly twice the speed of Fixed K=64 (0.735 steps/sec) — because fewer neurons are transferred per step. Dynamic K at 10% is the slowest MEFT condition at 0.500 steps/sec due to the larger K=102 transfer volume. This demonstrates that Dynamic K selection can recover a significant portion of MEFT's speed penalty relative to Fixed K=64 by reducing K.

Accuracy results show a consistent monotonic relationship with K. Exact Match increases from 0.000 at K=30 to 0.060 at K=102. Containment Match increases from 0.340 at K=30 to 0.400 at K=102. Notably, Dynamic K at 3% achieves Containment Match of 0.340 while being substantially faster (1.462 vs. 0.735 steps/sec) and more memory-efficient than Fixed K=64 (0.300 Containment). This suggests that Dynamic K scaling preserves more

information per transferred neuron than fixed K, since the same 3% of rank-1024 neurons (30 neurons) captures different information than an arbitrary fixed 64.

The most favorable operating point depends on the application constraint. If training speed is the primary concern, Dynamic K at 3% provides a 2× speedup over Fixed K=64 with a slight accuracy improvement. If accuracy is the primary concern, Dynamic K at 10% provides the best results (EM=0.060, Containment=0.400) at the cost of additional VRAM. Fixed K=64 sits between these extremes but is not Pareto-optimal: both the 3% and 10% configurations outperform it on at least one axis without sacrificing another.

7. Discussion

The results of Project 2 confirm the practical utility of MEFT's memory savings. Extension 1 shows that MEFT's ability to scale adapter rank beyond the baseline's VRAM limit translates into a 31% relative improvement in Containment Match (0.420 vs. 0.320) within the same 4 GB hardware constraint. This directly answers the question left open by Project 1: the memory savings are meaningful because they enable larger, more capable adapters.

The counterintuitive decline in the baseline's Containment Match from rank 256 to rank 2048 (0.320 to 0.220) warrants further investigation. One hypothesis is that larger GPU-resident adapters introduce training instability within the 300-step budget, as the adapter must learn from a cold start with more parameters. MEFT's sparse updates, which touch only K=64 neurons per step, may act as an implicit form of sparse regularization that stabilizes training at large ranks. This hypothesis would require controlled experiments with longer training runs to confirm.

Extension 2 demonstrates that calibration-based Dynamic K selection is a practical improvement over Fixed K=64. The results show a clear Pareto frontier: Dynamic K at 3% improves both speed and Containment Match relative to Fixed K=64, while Dynamic K at 10% provides the highest accuracy when extra VRAM is available. The monotonic relationship between K and accuracy suggests that K selection can be treated as a continuous accuracy-efficiency dial rather than a binary choice.

A key limitation of this study is the small training scale. With only 100 steps on 500 SQuAD examples, the absolute EM scores are low and the differences between conditions may not reflect the full benefit of larger adapters. Longer training runs (2000–5000 steps on the full SQuAD training set) would likely amplify the accuracy differences observed here and provide a more definitive comparison. This is the primary direction for future work.

8. Future Work

Longer training runs. The most immediate extension is to train for more steps (2000–5000) on the full SQuAD training set. This would provide more reliable accuracy estimates

and likely amplify the differences between rank conditions, giving a clearer picture of whether MEFT's larger adapters provide sustained accuracy benefits.

MoE routing optimization. The Key-Experts mechanism from the original MEFT paper was not implemented in this project. Adding MoE routing would reduce CPU-side TopK computation from O(dNM) to O(dM√r), potentially doubling training speed and addressing the primary throughput bottleneck identified in both projects.

Improved Dynamic K calibration. Extension 2 used a short calibration pass with per-layer forward hooks. Future work could improve this by using a larger calibration set, testing different activation thresholds, and recalibrating K during training instead of only once before training.

Asynchronous CPU-GPU transfer. Current transfers are synchronous, causing GPU idle time during the scoring and transfer steps. CUDA stream-based prefetching — overlapping layer i parameter transfer with layer i-1 GPU computation — could recover significant training throughput without any algorithmic changes.

9. LLM Assistance

Claude (claude-sonnet-4-6) was used throughout this project for planning, code generation, and writing assistance. Claude helped identify the correct evaluation approach for SQuAD with a causal language model (generation-based rather than span-extraction), and flagged an error in the original proposal code that incorrectly used start_logits and end_logits from AutoModelForQuestionAnswering. Claude also generated the preliminary code skeleton for train_meft_squad_json.py / train_meft_dynamic_k_json.py based on descriptions of the required functionality, which was then reviewed, debugged, and extended. Claude was additionally used to assist with grammar and punctuation in this report.

10. References

[1] Hao, J., Sun, W., Xin, X., Meng, Q., Chen, Z., Ren, P., & Ren, Z. (2024). MEFT: Memory-Efficient Fine-Tuning through Sparse Adapter. In Proceedings of ACL 2024, pp. 2375–2388.

[2] Rajpurkar, P., Zhang, J., Lopyrev, K., & Liang, P. (2016). SQuAD: 100,000+ Questions for Machine Comprehension of Text. EMNLP 2016.

[3] Hu, E. J., et al. (2022). LoRA: Low-Rank Adaptation of Large Language Models. ICLR 2022.

[4] He, J., Zhou, C., Ma, X., Berg-Kirkpatrick, T., & Neubig, G. (2022). Towards a Unified View of Parameter-Efficient Transfer Learning. ICLR 2022. [Parallel Adapter]

[5] Ren, J., et al. (2021). ZeRO-Offload: Democratizing Billion-Scale Model Training. USENIX ATC 2021.

[6] Wolf, T., et al. (2020). HuggingFace Transformers: State-of-the-Art Natural Language Processing. EMNLP 2020.